

Institutional Trading System Audit - Inconsistency Analysis

Critical Inconsistencies

1. Execution Lifecycle Contradiction

Issue: The document states execution flow as `CREATE → SEND → ACK → FILLED → RETRY`, but the Issues section says "No execution lifecycle".

Inconsistency: A defined flow suggests lifecycle tracking exists, but the issues explicitly contradict this.

Impact: Unclear whether partial lifecycle tracking is implemented or completely absent. This creates ambiguity in understanding the actual system state.

2. Risk Management Coverage Gap

Issue: Risk section mentions `if drawdown > threshold: stop()` implementation, yet states "Missing kill switch and capital allocation".

Inconsistency: A `stop()` mechanism exists, but a "kill switch" is listed as missing. These are functionally similar concepts—a kill switch is a hard stop mechanism.

Impact: Is there a soft drawdown stop or only a hard kill switch needed? The distinction is critical for risk management. Current implementation may be insufficient.

3. Architecture vs. Implementation Mismatch

Issue: Architecture diagram shows `Market Data → Strategy → Execution → Broker → Dashboard` (clearly structured flow), yet Issues section claims "Monolithic system" with "lacks separation".

Inconsistency: A logical separation in the architecture diagram contradicts the assertion that the system lacks separation of concerns.

Impact: Either the architecture is aspirational (not implemented) or the separation exists but is poorly enforced. Code quality and maintainability implications differ significantly.

4. Database Persistence Contradiction

Issue: Listed as "No DB persistence" under Issues, but Infra section mentions "AWS/OCI deployment" which typically implies database infrastructure.

Inconsistency: Cloud deployment suggestions without database persistence is operationally unrealistic for institutional trading (auditability, compliance, recovery).

Impact: Either the audit is incomplete or the infrastructure recommendations don't account for stated constraints.

5. Strategy Quality vs. Severity Ranking

Issue: Strategy section criticizes both ANN and Grid approaches:

- ANN: "too small, lacks features"
- Grid: "risky in trends"

Inconsistency: Yet the Roadmap shows Phase 1 as "Stability" without mentioning strategy redesign as a prerequisite. These are fundamental strategy problems that affect stability.

Impact: Confusing priority order. Strategy risk directly impacts system stability.

6. Broker Reconciliation Not Mentioned in Roadmap

Issue: Issues list "No broker reconciliation" as a critical gap, but the 4-phase roadmap (Stability → Correctness → Performance → Alpha) doesn't explicitly address it.

Inconsistency: A major operational risk is identified but not mapped to the remediation roadmap.

Impact: Unclear when (or if) this critical audit finding will be addressed. Broker reconciliation is necessary for correctness and capital safety.

7. Dashboard Functionality vs. System State

Issue: Dashboard section requires "WebSocket, live PnL, logs", implying a functioning backend. Yet the system is described as "Prototype stage" with missing execution lifecycle.

Inconsistency: Cannot provide live PnL from a system that lacks execution lifecycle tracking.

Impact: Dashboard requirements assume more advanced system capabilities than actually exist.

8. Hardcoded Configs Contradiction

Issue: Listed as an issue ("Hardcoded configs"), but no solutions mentioned in Roadmap or Infra sections.

Inconsistency: Configuration management isn't addressed in the improvement plan.

Impact: Environmental deployability (dev, test, prod) remains unresolved.

9. Async Processing Gap

Issue: Infra section recommends "async processing" and "low latency routing", but Issues don't explicitly mention synchronous bottlenecks.

Inconsistency: The infrastructure prescription doesn't clearly map to identified execution issues.

Impact: Either the audit missed identifying the core performance problems, or recommendations are speculative.

10. Retry System vs. Execution Lifecycle

Issue: Execution Engine states CREATE → SEND → ACK → FILLED → RETRY suggesting retry handling exists, but Issues say "Missing execution lifecycle".

Inconsistency: A retry state in the flow contradicts claims of missing lifecycle.

Impact: Fundamental confusion about what execution state management actually exists.

Summary Table

Issue	Stated	Contradicts	Severity
Execution Lifecycle	Flow defined → Missing	Critical clarity gap	High
Kill Switch	stop() exists → Missing	Functional definition unclear	High
Separation of Concerns	Diagram shows → Monolithic	Architecture/implementation mismatch	High
DB Persistence	Missing → AWS deployment	Operationally inconsistent	High
Strategy Risk	Critical flaws → Phase 1 not strategy	Roadmap priority mismatch	Medium
Broker Reconciliation	Critical gap → Roadmap omits	Implementation plan unclear	High
Dashboard	Requires live PnL → Prototype stage	Technical feasibility unclear	Medium
Hardcoded Configs	Issue listed → Unaddressed in plan	Incomplete remediation	Medium

Issue	Stated	Contradicts	Severity
Async Processing	Recommended → Root cause vague	Diagnosis unclear	 Medium

Recommendations

1. **Clarify Execution State Machine:** Define exactly what state tracking exists vs. what's missing. Create a detailed state diagram.
2. **Separate Architecture from Implementation:** Create two documents—one for the target architecture, one for current implementation gaps.
3. **Map Issues to Roadmap:** Each critical issue should have a specific phase assignment with clear acceptance criteria.
4. **Define Capital Safety Baseline:** Before Phase 1 (Stability), establish minimum requirements for:
 - Broker reconciliation
 - Kill switch functionality
 - DB persistence
5. **Validate Strategy Assumptions:** Specify which strategy (ANN vs. Grid) will be used and when redesign happens relative to other phases.
6. **Environmental Configuration:** Add Phase 0 for infrastructure and configuration management if not covered in Stability phase.